BTRFormer: Hierarchical Learning of Encrypted Traffic Using a Masked Autoencoder with Block-Based Traffic Representation

Junnan Yin[1,2,*], Lei Cui[2,*], Zhiyu Hao[2,†], Jiawei Sun[3], Peng Liu[4], and Xiaochun Yun[2,†]

[1]School of Cyberspace Security, Beijing Institute of Technology, Beijing, China, 3120245901@bit.edu.cn
[2]Zhongguancun Laboratory, Beijing, China, {yinjn2023,cuilei,haozy,yunxiaochun}@zgclab.edu.cn
[3]Beijing Research Institute of Telemetry, Beijing, China, sunjiaw@gmail.com
[4]Guangxi Normal University, Guangxi, China, liupeng@gxnu.edu.cn

2025 IEEE 33rd International Conference on Network Protocols (ICNP) | 979-8-3315-0376-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICNP65844.2025.11192383

***Abstract*—Encrypted traffic classification (ETC) is essential for ensuring network security and efficient management. Despite advances in deep learning, ETC remains challenging as existing models struggle to learn robust, discriminative representations from content-encrypted, highly imbalanced traffic.**

To address these challenges, we propose BTRFormer, a novel ETC approach that capitalizes on the inherent properties of encryption algorithms to enhance classification accuracy. At the core of BTRFormer lies a block-based, multi-layer traffic representation that adopts a 4×4 block as the fundamental unit, inspired by the encryption algorithm's use of 16-byte blocks for encryption operations. This representation preserves the intrinsic structure of encrypted payloads, facilitating the model's ability to learn deep semantic features. Subsequently, a transformer-based model is employed to learn from the multi-layer representation, capturing intra-block, inter-block, and inter-packet dependencies through block-wise attention mechanisms. Finally, BTRFormer leverages a pre-training phase on large-scale unlabeled data, followed by fine-tuning with a minimal amount of labeled samples to improve generalization and adaptability. Experimental results show that BTRFormer significantly outperforms SOTA methods on six real-world datasets, highlighting its effectiveness in encrypted traffic classification and secure network management.

***Index Terms*—Encrypted Traffic Classification, Transformer, Masked AutoEncoder, Pre-training, Fine-tuning**

I. Introduction

Network traffic classification is a key technology for identifying and distinguishing various applications and services within traffic, enabling efficient network management and security protection [1], [2]. In recent years, traffic encryption protocols such as TLS and QUIC has been widely adopted to enhance user privacy and anonymity on the Internet. However, encryption also poses challenges for traffic classification, as traditional classification methods that rely on packet content features become ineffective when applied to encrypted traffic. Furthermore, malicious actors can evade detection by employing privacy-enhancing technologies such as VPNs and Tor [3]. These challenges motivate us to explore how to capture implicit and robust patterns in encrypted traffic and to develop accurate and generalizable traffic classifiers.

Early traffic classification methods assume that different network services typically use distinct protocols and ports for communication, making it possible to distinguish services based on these features [4] (e.g., 5900 for VNC, 3306 for MySQL). However, with the diversification of nowadays network services and the widespread adoption of traffic encryption technologies, this approach faces significant challenges such as reduced accuracy and limited adaptability to complex network environments. In addition, machining learning (ML)-based methods typically rely on statistical features, such as packet sizes and inter-arrival times, to classify traffic. These features capture fundamental patterns and have shown effectiveness [5]–[7]. However, these methods rely on expert-designed features, which may fail to capture subtle or complex features and limit classification performance [8].

Deep learning (DL)-based methods can automatically learning intrinsic features directly from raw traffic, which are often difficult to extract manually, thereby achieving promising results [9]–[11]. However, these methods require large labeled datasets, making the process labor-intensive and time-consuming. With the growing popularity of the pre-training and fine-tuning paradigm, many methods have begun applying it to traffic classification [12]–[16]. They first pre-train on a large corpus of unlabeled traffic data to obtain generalized representations, and then fine-tune on a smaller set of labeled data to produce a classification model. Despite their performance, however, pre-trained models struggle to effectively learn from encrypted traffic, as the payload is predominantly encrypted.

In this paper, we propose BTRFormer, a novel model that exploits knowledge of encryption algorithms to enhance encrypted traffic classification capabilities. Specifically, traffic is first represented as an image-based multi-layer matrix, where each 4×4 block mimics the 16-byte group in the AES block structure [1], preserving the inherent structure and

∗ co-first author, † corresponding author.

979-8-3315-0376-5/25/$31.00 ©2025 IEEE

[1]AES (Advanced Encryption Standard) is one widely used encryption algorithm in TLS.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

encryption patterns within the traffic flow. Then, to capture intra-block features, inter-block dependencies, and inter-packet correlations across multiple layers, we propose a Transformer-based architecture that leverages a multi-layer attention mechanism for comprehensive representation learning. This enables the model to effectively learn the relationships between 16-byte blocks within individual packets and across the entire traffic trace, thereby capturing and contextualizing meaningful patterns from encrypted traffic. Finally, we employ the Masked AutoEncoder (MAE) to train our classifier through a two-stage process: pre-training and fine-tuning. In pre-training, we propose a mask task to learn relevant information by randomly masking bytes and reconstructing them with a lightweight decoder, which can gain a deeper understanding of traffic structures and content. In fine-tuning, we fine-tune the pre-trained model on a small amount of task-specific labeled data to improve the classification performance in specific scenarios.

We evaluate BTRFormer on six public traffic datasets. Experimental results demonstrate that BTRFormer consistently outperforms existing methods. For instance, it achieves improvements of 20.8%, 35.6%, and 4.4% in accuracy over three SOTA pre-training methods on the ISCXTor2016 dataset.

To summarize, this paper makes the following contributions:

- First, we propose a block-based, multi-layer traffic representation matrix inspired by the grouping principles of the AES encryption algorithm. To the best of our knowledge, the first attempt to classify encrypted traffic while explicitly modeling encryption structures.
- Second, we propose a Transformer-based framework with three layers of attention, capable of capturing intra-block, inter-block, and inter-packet relationships.
- Finally, we conduct extensive experiments on multiple public traffic datasets, covering tasks such as encrypted application classification, malicious traffic detection, and VPN-based traffic identification. Part of code is available at https://github.com/yyyjn/BTRFormer.

II. Related Work

Rule-based Methods. Early traffic classification methods primarily relied on rule matching, using basic features such as port numbers and communication protocols to identify traffic types [17]–[20]. However, as network services have become increasingly diverse and complex, rule-based methods have proven less effective for encrypted traffic. Simple port numbers or protocol features are no longer sufficient to characterize complex traffic patterns [4], [20]. In particular, with the widespread use of traffic mixing and camouflage techniques, traditional rule-based methods can be easily bypassed.

DPI-based Methods. Also known as signature-based identification, deep packet inspection (DPI) accurately classifies traffic by extracting and matching specific signatures, such as characters, strings, bit patterns, and symbols within the traffic content. For example, Wang et al. [21] propose the length-based matching (LBM) framework, which combines accelerated regular expression matching techniques with a dual finite automata structure, significantly improving both matching speed and memory efficiency. Similarly, Fernandes et al. [22] develop a lightweight DPI framework that reduces computational overhead while maintaining high precision. Although DPI addresses the limitations of port-based identification, it becomes ineffective when packets are encrypted.

Machining learning-based Methods. These methods [4]–[6], [23]–[26] typically rely on expert knowledge to design traffic features, such as packet size and time interval, which are then fed into machine learning models for classification. Common models include support vector machines (SVM), random forests (RF), and K-nearest neighbors (KNN). For example, AppScanner [4] employs random forests to classify traffic types based on statistical features, while CUMUL [5] selects 104 optimal statistical features as input to an SVM for website traffic identification. However, the performance of these methods is highly dependent on the quality of selected features and often struggles to adapt to varying scenarios. Additionally, it is also very difficult to design universal statistical features to adapt to the ever-changing network environment and application types, which limits the flexibility and adaptability of the methods.

Deep learning-based Methods. Deep learning can automatically extract complex features from data, eliminating the need for the tedious process of manual feature engineering. Typical methods [9]–[11], [27]–[34], [34]–[38], such as DF [27] and FS-Net [10], use CNN or RNN to extract representations from raw traffic data, while Deeppacket [28] and TSCRNN [29] process the raw payload for classification. However, the performance of deep learning models often depends on large-scale labeled data, and annotating traffic data is not only labor-intensive and time-consuming but also challenging to the difficulty of obtaining large amounts of real traffic samples. Furthermore, when packet bytes are directly used as input, the byte information in long packets can overshadow other important features, leading to unstable model performance.

Pretraining-based Methods. Pre-training leverages self-supervised learning on large unlabeled data to learn general feature representations, reducing reliance on labeled data. It typically involves two stages: pre-training on unlabeled data and fine-tuning on limited labeled data for specific tasks. In natural language processing, BERT [39] uses masked language model tasks (such as randomly masking part of the vocabulary and predicting) and next-sentence prediction to learn deep semantic representations. In traffic classification, He et al. [13] and Lin et al. [12] treat packet bytes as 'words' and apply BERT to pre-train for encrypted traffic classification. However, due to the lack of clear high-level semantic units in traffic bytes, such methods are difficult to completely migrate to the pre-training models designed for text processing. In computer vision, pre-training usually forms positive and negative sample pairs through data augmentation to learn features in a contrastive learning manner. For example, SimCLR [40] learns meaningful feature representations by maximizing the similarity between differently augmented views of the same image. However, for encrypted traffic, directly enhancing the packet bytes will damage the original information, making

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

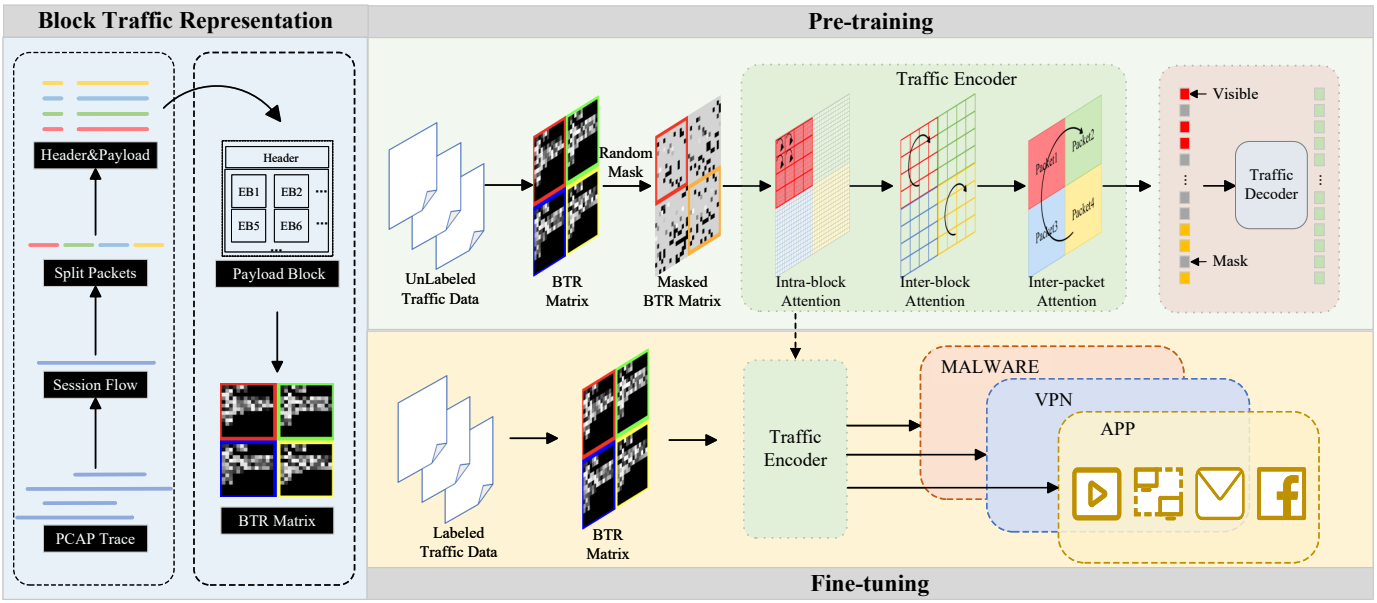


Fig. 1: The overview of BTRFormer framework.

such methods unsuitable. Alternatively, YaTC [41] tokenizes encrypted traffic into fixed-size patches and employs masked autoencoders (MAE) for self-supervised learning, effectively adapting vision techniques to encrypted traffic classification.

III. Methods

A. Motivation

The Advanced Encryption Standard (AES) is a widely adopted symmetric encryption algorithm in TLS, providing secure encryption for network traffic. AES operates on fixed-size data blocks of 16 bytes (128 bits). Each 16-byte block undergoes a series of transformations designed to ensure security by achieving confusion (obscuring the relationship between plaintext and ciphertext) and diffusion (distributing changes across the entire data block).

Using Cipher Block Chaining (CBC) mode as an example, as illustrated in Fig. 2, each plaintext block is first XORed with the ciphertext of the previous block (Stage①). It is then encrypted through multiple rounds, with each round involving four operations (Stage②): SubBytes, ShiftRows, MixColumns, and AddRoundKey. Notably, each encryption step depends on the ciphertext of the preceding block. Consequently, even a slight alteration in the plaintext will propagate changes to all subsequent ciphertext blocks, establishing dependencies between encrypted blocks.

From this algorithm, we identify two aspects that can be utilized to enhance classification models. First, the 16-byte block, which exhibits strong internal relationships among its bytes, can be represented as an independent semantic unit. Second, the inter-block relationships can be leveraged to learn higher-level structural patterns embedded in encrypted traffic. Note that some algorithms, such as GCM, lack direct inter-block dependencies; however, indirect dependencies still exist due to the shared cryptographic context, such as initialization vectors (IVs), keys, or internal state updates during encryption.

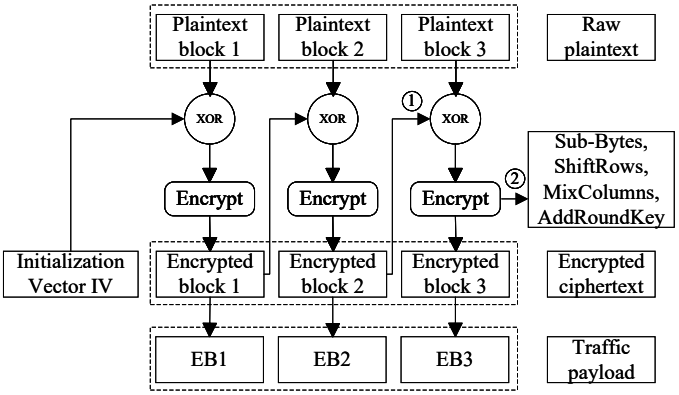


Fig. 2: The encryption mode of CBC.

B. Model Architecture

In this paper, we present BTRFormer, a novel framework designed to learn generalized and robust representations of encrypted traffic to improve classification performance. At the core of BTRFormer is a 16-byte block-based image representation for network traffic, which effectively preserves essential flow features within encrypted blocks, packets, and entire flows. Based on this BTR representation, a hierarchical learning approach is proposed to model the relationships at packet-level as well as at the flow-level, thus enabling a comprehensive capturing of traffic patterns. In addition, a two-stage training strategy is employed to further enhance the ability that learn valuable and and general information from large-scale unlabeled data and achieve superior performance even with limited labeled samples. More specifically, BTRFormer comprises of three primary components, as shown in Fig. 1.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

Block-Based Traffic Representation(BTR) Matrix. The BTR matrix models raw traffic data at multiple granularities, effectively capturing essential flow information in a structured format. Drawing inspiration from the AES algorithm's use of 16-byte blocks, each 16-byte segment of payload is represented as a 4×4 blocks during image construction, preserving key traffic features across encrypted blocks and packets.

Hierarchical Traffic Transformer. This module incorporates intra-block, inter-block and inter-packet attention mechanisms, allowing it to uncover subtle patterns and dependencies within individual blocks and across multiple blocks and packets. This design enhances the model's capability to efficiently analyze and represent features of encrypted traffic.

Two-Stage Strategy. This strategy adopts a two-stage training process: pre-training and fine-tuning. In the pre-training phase, random bytes within the BTR matrix are masked, and the model is tasked with reconstructing the original input. This enables the model to learn robust and generalizable representations from large-scale unlabeled data. In fine-tuning, the pre-trained parameters are transferred to downstream tasks, which are further adapted to perform task-specific classifications using only a small amount of labeled data.

C. Block-based Traffic Representation

In real-world network environments, large volumes of traffic often encompass multiple flows of diverse categories, such as different applications, protocols, or services, which make it difficult to learn a stable and discriminative representations for specific types of traffic. Many existing approaches extract a fixed number of raw bytes from each flow to form a two-dimensional matrix, subsequently transforming it into an image for classification using image processing algorithms. However, this method suffers two notable limitations: (1) Compared to the TLS protocol headers in the traffic, which contain plaintext information, the payload is encrypted using the AES algorithm. Therefore, existing image-based representation cannot reveal substantial structural or semantical information. (2) For certain flows[2], a large packet may fill the entire matrix, leaving no space for subsequent packets. This constraint limits the matrix's ability to represent more packets within the flow. These limitations highlight the need for a more advanced traffic representation method that can alleviate the semantic opacity of encrypted traffic while effectively accommodating multiple packets.

To this end, we propose BTR, an image-based, multi-level, encryption-aware representation of network traffic. As illustrated in Fig. 3, each instance of BTR is formulated as a two-dimensional matrix that captures three levels of traffic information: 1) Block-level: Each block consists of 16 bytes, arranged as a 4×4. This design is inspired by the 16-byte block structure of AES, providing encryption-aware granularity at the byte level. 2) Packet-level: Each packet is divided into two parts: a header matrix and a payload matrix. The header matrix

[2]A flow is defined as a series of packets that share the same IP address, port number, and protocol, following the traditional definition commonly adopted in prior literature.

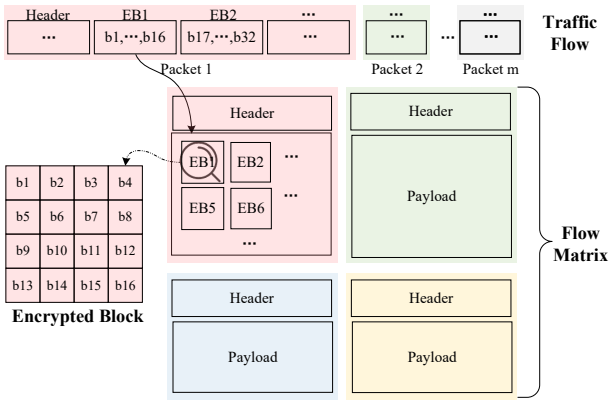


Fig. 3: The multi-layer traffic flow representation.

is constructed from protocol-specific header fields, including the IP header, TCP header, and other extension fields. The payload matrix contains multiple 4×4 blocks each recording 16 payload bytes. These two matrices are concatenated to form a packet-level matrix, as depicted in the Packet part of Fig. 3. 3) Flow-level: To capture the temporal structure of a flow, multiple packet-level matrices are arranged to form the final BTR matrix. This representation offers two advantages. First, it facilitates effective learning of semantic features within each 16-byte block during convolution. Second, it captures inter-block dependencies, which is consistent with the chained computation of ciphertext in AES.

More specifically, for each packet, we first extract the initial 64 bytes to construct the header matrix (4×16), which typically includes 20 bytes for the IP header, 20 bytes for the TCP header, and remaining bytes for other optional fields. Next, we extract 192 bytes from the payload of each packet, truncating those exceeding 196 bytes and zero-padding those shorter, to form a payload matrix of 12 rows by 16 bytes, resulting in a fixed-size 12×16 representation. These blocks are arranged in a column-first, then row-wise order to align with the AES encryption algorithm's processing mechanism, thereby preserving the structural and semantic integrity of the original traffic data as much as possible. As a result, each packet is uniformly represented as a fixed-size 16×16 matrix. At the flow level, we configure the BTR matrix to include four packets, i.e., $M = 4$, and arrange the corresponding packet matrices in a 2×2 grid. Consequently, the resulting flow-level matrix has dimensions 32×32, with both H and W equal to 32, in a total of 1024 bytes. As illustrated in Fig. 1 and 3, the upper-left corner (red area) represents the first packet, while the green, blue, and yellow areas denote the second, third, and fourth packets, respectively.

D. Hierarchical Traffic Transformer

Given that the proposed image-based representation exhibits hierarchical structures across blocks, packets, and flows, it is essential to perform multi-level feature extraction to fully capture the underlying traffic semantics. To this end, we introduce a three-layer attention module that learns traffic patterns at the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

level of individual encrypted blocks, inter-block relationships, and cross-packet dependencies. As a result, it provides a robust representation of traffic for effective classification.

1) Embedding Module for Encrypted Blocks: Before applying multi-layer attention, we first embed the 4×4 encrypted blocks. According to the characteristics of the AES encryption mechanism (16-bytes per group), the input BTR matrix $x \in \mathcal{R}^{H \times W}$ is first split into non-overlapping blocks by a block splitting module, which size of each block is 4×4, and the total number of blocks is $N = \frac{H}{4} \times \frac{W}{4}$. To learn more robust and generalizable representations from the encrypted traffic data, we further introduce a masking strategy to the module. Specifically, each block is split into non-overlapping patches, denoted as $x_p \in \mathcal{R}^{N \times 2^2}$, which are treated as 'tokens', with its feature represented by the concatenation of the raw traffic byte values. During the training stage, a certain percentage of tokens are randomly selected and masked. The model is then trained to predict the original bytes of these masked tokens using the surrounding unmasked context. This masking strategy forces the model to focus on learning the underlying structure and dependencies within the traffic data, thereby improving its ability to generalize to unseen or partially observed traffic patterns. As a result, the raw feature of traffic bytes is projected into a specified dimension (denoted as D). To maintain important structural information from the raw traffic data, the position embeddings(E_{pos}) are added to the embeddings as the input of the traffic encoder, i.e., $x_0 = \left[x_p^1 E; x_p^2 E; \ldots; x_p^N E\right] + E_{\text{pos}}$.

As a result, all 16 elements of one block represent the encrypted block of raw bytes, with each sub-matrix comprising one header row and three payload rows. Then, several Transformer modules with attention mechanisms are applied to these blocks, maintaining the number of blocks ($\frac{H}{4} \times \frac{W}{4}$).

2) Hierarchical Attention Module: The blocks within the same packet exhibit strong correlations, according to the AES mechanism in Fig 2. Leveraging this characteristic, we are inspired to design an in-packet attention module that automatically learns and captures the correlations between encrypted blocks, enabling more effective modeling of the timing and contextual information in conversation flows. Specifically, we develop a Transformer-based [42] traffic encoder that employs window-based multi-head self-attention to compute attention within each packet. This enables the encoder to learn both the feature and dependencies among encrypted blocks within a packet and the interactive relationships across packets.

Intra-block attention module. Each block consists of 16 bytes (4×4). To effectively learn the content and internal dependencies within a single encrypted block, we apply multi-head self-attention across its multi tokens. This mechanism enables efficient interactions among the encrypted bytes of the same block by leveraging the correlations captured across multiple attention heads, denoted as $\mathit{Concat}(\mathit{head}_1, \cdots, \mathit{head}_n)^2$, where each head is computed through the attention function, i.e., $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D_k}}\right)V$. Here, $Q = x_l W^q$, $K = x_l W^k$, $V = x_l W^v$, where W^q, W^k, W^v are learnable parameters and $\frac{1}{D_k}$ is the scaling factor.

Inter-block/Intra-packet attention module. At this stage, our goal is to learn the interaction of information within individual packets, leveraging the traffic characteristic that encrypted blocks within the same packet tend to exhibit stronger correlations. This enables the generation of higher-level representations. Specifically, for each packet, we treat it as a window composed of multiple encrypted blocks. To capture the relationships among these blocks, we apply multi-head self-attention across the blocks within the packet, rather than over the entire traffic trace. This enables effective interaction extraction among encrypted blocks by leveraging cross-head correlations to better model intra-packet dependencies. Each attention head is still computed using the above equation, with the difference lying in the input representation.

Computational complexity. Assuming each packet contains (M×M) blocks, the global MSA over the entire flow has complexity $\Omega(\text{In-flow}) = 4hwC^2 + 2(hw)^2C$, scaling quadratically with $h \times w$, while in-packet attention has $\Omega(\text{In-packet}) = 4hwC^2 + 2M^2hwC$, scaling linearly when M is fixed. Thus, global attention becomes impractical for large flows, whereas in-packet attention offers better scalability, enabling efficient handling of larger traffic volumes with the same resources.

Inter-packet attention module. As shown in Fig 3, the representation of a flow consists of multiple packet-level images stacked together. These images are uniformly partitioned into non-overlapping windows in the BTR matrix, with each window corresponding to a single packet.

In network flows, packets within the same session often exhibit various types of correlation. 1) For IP and TCP protocols, many fields in header remain consistent across packets. For example, fields such as source and destination IP addresses, ports, and protocol identifiers are typically constant or similar. This consistency enables the identification of packets as belonging to the same session, thereby establishing flow-level context. 2) TCP connections exhibit clear sequential behavior. For example, the packet sequence numbers generally increase in ascending order, and each acknowledgment number corresponds to the sequence number of the preceding packet. This ordering helps maintain and identify packet sequence integrity. 3) Packets within the same session also tend to share similar statistical and content-level characteristics. For instance, features such as packet size and inter-packet intervals often follow consistent patterns. These similarities contribute to establishing content-level associations across packets.

Thus, to capture flow-level features across packets, we introduce an inter-packet attention module that facilitates information transfer between packets. Specifically, we re-partition the BTR matrix such that each newly defined region contains features from adjacent packets, enabling each region to capture richer contextual information. This re-partitioning produces multiple windows of varying sizes and shapes. We then apply a shifting operation to these windows, allowing their boundary areas to overlap. This overlap promotes cross-packet information fusion and enhances the model's ability to capture inter-packet dependencies.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

Conclusion. Leveraging the aforementioned three-layer attention mechanism, the original BTR matrix is projected into vector representations that capture hierarchical traffic features and enable effective traffic classification, rather than operating raw bytes as Hi-Transformer [43].

E. Pre-training and Fine-tuning

1) Pre-training BTRFormer: In the pre-training stage, we adopt the MAE paradigm [44] to perform self-supervised learning on unlabeled data. This is achieved by applying a masking mechanism and employing an asymmetric encoder-decoder architecture to reconstruct the raw bytes in the BTR matrix. Specifically, we randomly mask a majority of the tokens and input only a small portion of the visible (unmasked) tokens into the model as part of the self-supervised task. The traffic encoder extracts meaningful representations from these visible tokens and generates corresponding encoder outputs. Then, a lightweight decoder combines these encoder outputs with the masked tokens to reconstruct the missing regions of the BTR matrix, thereby enabling the model to learn general traffic representations from unlabeled data. Compared to Flow-MAE's burst-level masking [45], the BTR-specific masking strategy leverages the inherent structure of the BTR matrix, resulting in enhanced robustness. The overall process is illustrated in the pre-training section of Fig. 1. The reconstruction loss used during training is defined as the mean squared error (MSE) between the real tokens y_{real} and the reconstruct tokens y_{rec}, i.e., $\mathcal{L} = \mathrm{MSE}(y_{rec}, y_{real})$.

The entire pre-training stage eliminates the need for heavily manual annotation and allows the use of large-scale unlabeled data collected from real-world scenarios. Notably, we observe that high masking ratio (e.g., 0.9) leads to improved performance in downstream tasks. However, such a high rate results in a limited number of visible tokens, reducing the amount of original information and making it difficult to capture dependencies both within and across packets. Fortunately, the in-packet and inter-packet attention modules effectively integrate contextual information, allowing the model to extract meaningful features at both the packet and flow levels.

After pre-training, the encoder learns general traffic features from large-scale unlabeled data, thereby enhancing the model's generalizability and adaptability. This provides a robust foundation for subsequent fine-tuning.

2) Fine-tuning BTRFormer: Compared to task-independent pre-training, which can be applied to any type of traffic representation, fine-tuning is more effective in serving downstream classification tasks. Furthermore, since the architectures of the pre-training and fine-tuning stages are basically the same, the encoder parameters obtained during pre-training can be seamlessly transferred to the packet-level and inter-packet attention modules to facilitate hierarchical feature extraction at both the packet and flow levels. For classification tasks, we first transform the task-specific traffic data into a BTR matrix, which is then fed into the pre-trained BTRFormer. All parameters are subsequently fine-tuned to optimize performance on the specific downstream task.

IV. Experiments

We implement BTRFormer based on Pytorch 1.13.1 with Python 3.9 and conduct the experiments on a server configured with 1 NVIDIA A100 80GB GPU, 28-core Intel(R) Xeon(R) 2.00GHz CPU, and Ubuntu-20.04.1. We evaluate BTRFormer on multiple datasets to demonstrate:

- BTRFormer is able to perform well on several different traffic classification tasks.
- BTRFormer achieves superior performance in traffic classification tasks compared to existing ML-, DL-, and Pretraining-based methods.
- The impact of key modules on classification performance.

A. Experiment Settings

1) Datasets: To evaluate the effectiveness of the BTRFormer, we conduct experiments on six public real-world encrypted traffic datasets, including encrypted traffic, anonymous traffic, IoT traffic, malware traffic, as listed below:

ISCXVPN2016/ISCXNonVPN2016 [46] is a benchmark dataset for traffic classification, including multi-application traffic in both VPN and non-VPN scenarios to support analysis of protocol obfuscation and encrypted communication. The VPN traffic includes 2,773 flows across 7 application classes, while non-VPN traffic comprises 6,231 flows across 6 classes.

ISCXTor2016 [47] is a benchmark dataset for classifying encrypted Tor traffic, containing 8 categories such as chat, email, and others. Since its non-Tor traffic overlaps with ISCXVPN2016, we focus on Tor-specific traffic to evaluate encrypted application classification on the Tor network.

CICIoT2022 [48] is a recent benchmark for IoT device behavior analysis, including network traffic from various devices using protocols like IEEE 802.11. We use a subset containing seven benign and three attack traffic types, with 7,998 flows for training and 4,002 flows for testing.

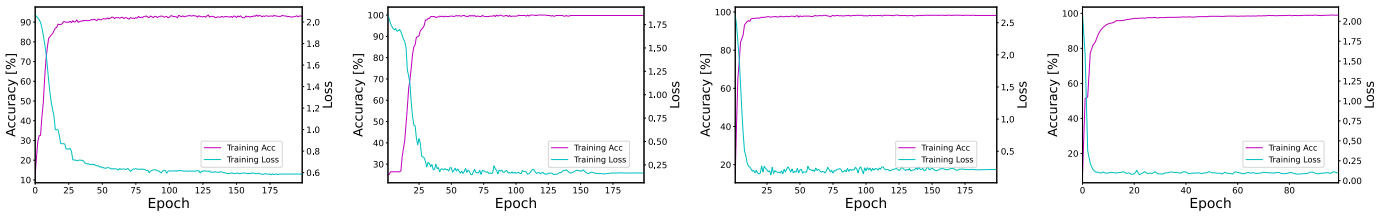
USTC-TFC2016 [49] is a well-known benchmark for distinguishing benign and malicious network traffic, containing 10 categories each of benign and malware traffic. We select a subset with 4006 flows for training and 994 flows for testing.

CSTNET-TLS1.3 [12] is a large-scale encrypted traffic dataset collected under CSTNET, with traffic fully encrypted using TLS 1.3 protocol. We select a subset of the dataset for evaluation, consisting of traffic from 68 applications, with 26,119 flows used for training and 6,559 flows for testing.

More specially, each dataset is split into a training dataset and a testing dataset with a 4:1 ratio. In the pre-training stage, the four training datasets (ISCXVPN2016, ISCXTor2016, CICIoT2022, and USTC-TFC2016) are combined to construct a large-scale unlabeled dataset for pre-training. In the fine-tuning stage, the classifier performs supervised learning on the corresponding traffic classification tasks. Additionally, ISCXNonVPN2016 and CSTNET-TLS1.3 are used for transfer learning experiments in Section IV.F.

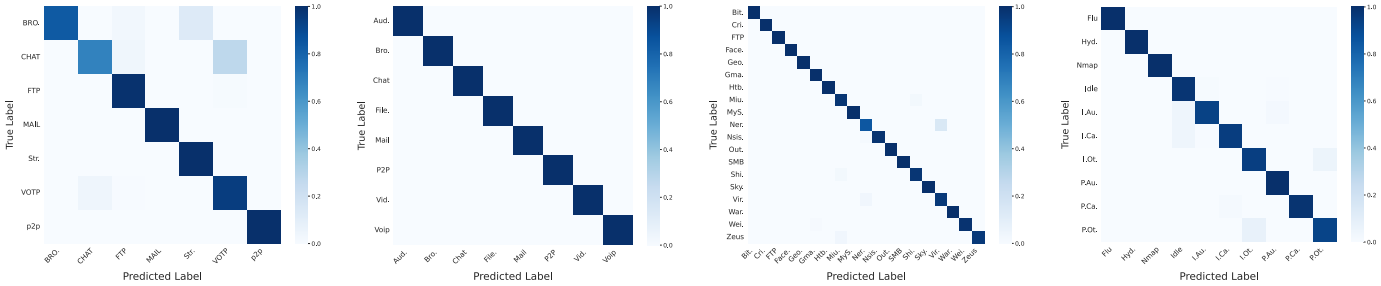
2) Data Pre-processing: For data preprocessing, we first split the raw traffic into flows based on IP addresses, port numbers, and protocol types. Next, we remove packets that are irrelevant to the transmitted content, such as ARP and DNS

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.



(a) ISCXVPN2016 (b) ISCXTor2016 (c) USTC-TFC2016 (d) CICIoT2022

Fig. 4: The training accuracy and loss of BTRFormer.



(a) ISCXVPN2016 (b) ISCXTor2016 (c) USTC-TFC2016 (d) CICIoT2022

Fig. 5: The confusion matrices of BTRFormer.

packets. To mitigate potential bias from strongly identifying information in packet headers, such as IP addresses and port numbers [28], [49], [50], we strip Ethernet headers, set all port numbers to zero, and replace IP addresses with random values while preserving their directionality. Finally, we keep only the first four packets from each flow.

3) Model configuration: For pre-training, we use the samples from all four datasets, with a batch size of 256 and a total of 150,000 training steps. The AdamW optimizer is adopted, with a base learning rate of 0.001 and a masking rate of 0.9. For fine-tuning, the model is independently trained on each dataset to perform specific tasks, using a batch size of 128, a learning rate of 1.25e-3, and a total of 200 epochs.

4) Evaluation Metrics: We use accuracy (acc) and F1 score ($f1$), to measure model performance, as adopted in many ETC studies [10], [12], [13], [27], [41], [49], [51].

B. Classification Performance of BTRFormer

To comprehensively evaluate the classification performance of BTRFormer, we analyze its training accuracy and loss on four datasets during the fine-tuning stage. As shown in Fig.4 , it can be seen that our model effectively extracts feature information from the training datasets, achieving high training accuracy and low loss. As the number of epochs increases, the training loss gradually stabilizes, indicating that the model has converged. Notably, Fig. 4d shows that BTRFormer achieves convergence within 10 epochs on CICIoT2022, demonstrating that our method can efficiently complete the training process.

In addition, we analyze the confusion matrices of the classification results on the four test datasets. As shown in Fig.5, BTRFormer achieves high accuracy across all four traffic classification tasks, demonstrating its strong generalization capability. For the ISCXTor2016, USTC-TFC2016, and CICIoT2022 datasets, BTRFormer accurately distinguishes between different traffic types, achieving nearly 100% classification accuracy for each class. However, for the ISCXVPN2016 dataset, some misclassifications are observed between CHAT and VOIP traffic. According to our analysis, VOIP and CHAT traffic originate from the same set of applications, including Facebook, Hangouts, and Skype. Consequently, they exhibit similar traffic patterns, and this overlap is a key factor contributing to misclassifications and lower accuracy.

C. Comparison with State-of-the-Art Methods

To perform a comprehensive evaluation of our proposed method, we compare BTRFormer with several state-of-the-art methods, which fall into the following three categories:

- **ML-based methods**: FlowPrint [7] and AppScanner [4], which rely on handcrafted statistical features to classify.
- **DL-based methods**: DF [27], FS-Net [10], 2DCNN [49], 3DCNN [51], and Deeppacket [28], which use raw packet information for supervised learning and automatically extract complex features through deep learning models.
- **Pretraining-based methods**: PERT [13] and ET-BERT [12], which treat traffic feature extraction as a pre-training task within natural language processing paradigm, while YaTC [41] adopts an image-based pre-training framework. These methods are subsequently fine-tuned on limited labeled data to enhance classification performance.

Table I presents a comparison of various models on encrypted traffic classification tasks. The results clearly show that deep learning (DL)-based methods generally outperform traditional machine learning (ML)-based methods that rely on handcrafted statistical features, demonstrating the superiority

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Performance comparison of different models on four public datasets.

Method	ISCXVPN2016		ISCXTor2016		USTC-TFC2016		CICIoT2022	
	acc	$f1$	acc	$f1$	acc	$f1$	acc	$f1$
FlowPrint	31.3%	16.62%	27.60%	11.92%	27.40%	12.10%	54.40%	47.17%
AppScanner	78.80%	77.95%	47.90%	45.89%	56.30%	54.47%	81.20%	71.08%
DF	58.30%	56.41%	35.90%	26.54%	54.60%	51.80%	63.40%	60.66%
FS-Net	86.40%	84.31%	56.60%	51.46%	87.60%	85.39%	89.90%	89.55%
2DCNN	80.80%	80.71%	30.10%	17.82%	92.10%	92.03%	89.30%	89.18%
3DCNN	80.10%	79.71%	38.00%	30.10%	92.50%	92.37%	87.50%	87.63%
Deeppacket	84.40%	83.91%	33.10%	25.46%	85.60%	85.33%	87.50%	87.36%
PERT	88.80%	84.40%	79.20%	77.95%	95.80%	95.83%	91.90%	91.74%
ET-BERT	86.60%	86.33%	64.40%	63.57%	96.80%	96.87%	92.05%	92.09%
YaTC	94.05%	**93.89%**	95.60%	95.66%	85.85%	85.81%	95.67%	95.67%
BTRFormer (Our)	**94.13%**	93.85%	**100.00%**	**100.00%**	**98.40%**	**98.40%**	**99.13%**	**99.13%**

of deep learning in extracting complex features. Among DL-based methods, DF [27] experiences the worst performance, primarily because it uses only the packet direction as input, which limits its ability to learn discriminative features. Moreover, DL-based methods perform relatively poorly on the ISCXTor2016 dataset. This can be attributed to the nature of Tor traffic, which is encrypted across multiple layers, exhibits variable packet lengths, and is often adversarially obfuscated. These characteristics make it challenging for traditional feature extractors to identify meaningful traffic patterns. In contrast, our BTR matrix can capture the intrinsic relationships among packets, leading to higher classification accuracy.

In addition, pretraining-based methods achieve better results than standard DL-based approaches, as pre-training enables models to learn latent feature from large-scale unlabeled data. Notably, PERT [13], ET-BERT [12], YaTC [41] and our method consistently outperform other approaches, illustrating that the pre-training strategies are well-suited for downstream tasks in encrypted traffic classification. Specifically, our method and YaTC significantly outperform other pre-trained models like ET-BERT and PERT. The improvement stems from the fact that ET-BERT and PERT treat encrypted traffic as natural language and attempt to extract semantic information from byte sequences. However, encrypted payload consists of seemingly meaningless byte streams, making such text-based approaches less effective in capturing underlying structural patterns. In contrast, our method transforms encrypted traffic into a structured, image-based representation aligned with encryption block boundaries, enabling more effective feature extraction and yielding substantial performance gains. Compared to YaTC, which also applies byte masking to image-based representations, BTRFormer achieves superior performance, for example, an average improvement of 5.09% in $f1$ score. We attribute this gain to the combination of the BTR representation and multi-layer attention, where the former captures structural encryption semantics, while the latter learns hierarchical semantic dependencies. Together, they enable BTRFormer to extract richer features than YaTC.

In conclusion, our model BTRFormer achieves superior performance in terms of accuracy and $f1$ score across all datasets, highlighting its effectiveness in encrypted traffic classification. Specifically, our method achieves an accuracy of 94. 13% on ISCXVPN2016, 100. 00% on ISCXTor2016, 98. 40% on USTC-TFC2016 and 99. 13% on CICIoT2022. We suspect that the effectiveness stems from the imperfect randomness of encrypted payloads, as real-world ciphers often deviate from ideal entropy. As revealed by ET-BERT, the Matrix Rank test yields low p-values for common ciphers, AES-GCM (0.5674), AES-CBC (0.4890), where 1 indicates perfect randomness. Notably, our approach relies solely on the first four packets of each flow, enabling early-stage decision-making. This is particularly valuable in time-sensitive scenarios where rapid traffic classification is essential.

D. Ablation Study

In this section, we conduct an in-depth analysis of the key components of BTRFormer to explore the gain of each part to the overall performance.

1) Gain of Pre-Training Tasks: Table II presents the performances that the model without pre-training on four specific traffic classification tasks, i.e., fine-tuning starting directly from a randomly initialized model. Among them, the hyper-parameters of the model without pre-training are set same as the pre-training stage. It can be seen that the largest decreases on $acc(f1)$ are ISCXVPN2016 and ISCXTor2016 datasets, which decrease by 11.71%(12.27%) and 17.80%(17.91%), respectively. This indicates that the model fails to learn useful information without pre-training, which may be the following reasons: (i) The number of samples in these two datasets is relatively small (fewer than 3,000) compared to the other two datasets (USTC-TFC2016 and CICIoT2022). The limited amount of training data hinders the model's ability to learn important information between samples, leading to the poor performance. (ii) Both two datasets present a challenge of class imbalance, which is particularly pronounced in ISCXVPN2016, leading to biased model performance where the majority classes dominate predictions while the minority classes are often misclassified or overlooked. (iii) Both two datasets use encryption or obfuscation techniques, which makes traffic analysis considerably more challenging. Therefore, in scenarios where downstream task data is scarce or the categories are unbalanced, pre-training is very necessary, as it can learn basic traffic information, which can be quickly transferred to downstream tasks. Furthermore, $acc(f1)$ of the USTC-TFC2016 and the CICIoT2022 dataset decreased slightly, by 1.50% (1.55%) and 2.21% (2.07%) respectively.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Classification performance without pre-training.

Datasets	acc	$f1$
ISCXVPN2016	82.42% (↓ 11.71%)	81.58% (↓ 12.27%)
ISCXTor2016	82.20% (↓ 17.80%)	82.09% (↓ 17.91%)
USTC-TFC2016	96.90% (↓ 1.50%)	96.85% (↓ 1.55%)
CICIoT2022	97.10% (↓ 2.21%)	97.06% (↓ 2.07%)

2) Gain of BTR Representation: To explore the effect of BTR on the performance of traffic classification tasks, we conduct experiments without BTR and pre-training on four datasets, as shown in Table III. Specifically, we directly use the raw traffic bytes without BTR. The first 1024 bytes of each flow are extracted and reshaped into a 32×32 image as input to the Transformer architecture. From Table III, it can be seen that $acc(f1)$ decreases by 16.23% (17.12%), 65.28% (71.74%), 3.40% (3.45%), and 3.23% (3.27%) on the ISCXVPN2016, ISCXTor2016, USTC-TFC2016, and CICIoT2022 datasets, respectively. The results indicate that the BTR matrix representation is effective in improving feature extraction, further enhancing the model's classification performance due to the 16-bytes (4×4) block.

TABLE III: Performance without BTR and pre-training.

Datasets	acc	$f1$
ISCXVPN2016	77.90% (↓ 16.23%)	76.73% (↓ 17.12%)
ISCXTor2016	34.72% (↓ 65.28%)	28.26% (↓ 71.74%)
USTC-TFC2016	95.00% (↓ 3.40%)	94.95% (↓ 3.45%)
CICIoT2022	95.90% (↓ 3.23%)	95.86% (↓ 3.27%)

3) Gain of Masking: To explore how the masking rate impacts the learned representations, we conduct experiments with different masking rates (0.25, 0.5, 0.75, and 0.95, respectively) on all four datasets, as illustrated in Fig. 6. Specifically, we select the pre-trained models at the 150,000th steps to evaluate, and set the learning rate of the fine-tuning to 1.25e-3. It can be seen that a higher masking rate tends to enhance model performance, with most data achieving the highest $f1$ score at a masking rate of 0.75. However, excessively high masking rates (e.g., 0.95) make the reconstruction task overly challenging, resulting in a significant drop in performance. We represent traffic as images, and prior studies [44], [52] have shown that image tasks can tolerate higher masking ratios than language tasks. This aligns with our results that traffic classification exhibits greater information redundancy than natural language processing, with higher masking ratios yielding better performance. In addition, since encrypted traffic bytes resemble pixels in an image and lack clear semantic units, using the image-based BTR for traffic representation is reasonable. This also demonstrates that traffic bytes can be effectively classified and analyzed without complete semantic understanding, highlighting the advantage of using a high masking rate in self-supervised learning for this task.

E. Effects of Various Settings

In this section, we explore the impact of the factors by configuring BTRFormer with different settings.

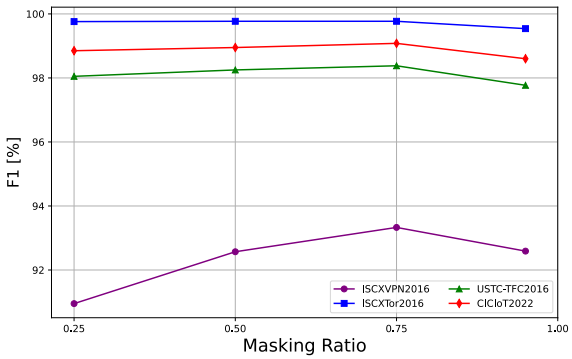


Fig. 6: Performance($f1$) with varying masking ratios.

1) Pre-training Steps: Fig. 7 shows how pre-trained models from different pre-training steps affect the performance of downstream traffic classification tasks. Specifically, we select the models at the 30,000th, 60,000th, 90,000th, 120,000th, 150,000th, 180,000th and 200,000th pre-training steps on the all four datasets, with a fine-tuning learning rate of 1.25e-3. It can be seen that, without pre-training, the fine-tuning performance on ISCXVPN2016 and ISCXTor2016 is relatively poor. When the number of pre-training steps reaches 30,000, the $f1$ improves rapidly. However, once the number of pre-training steps exceeds 60,000, further improvements in the $f1$ are marginal. For example, on the CICIoT2022 dataset, the $f1$ at 60,000, 90,000, and 120,000 steps are 98.38%, 98.49%, and 98.55%, respectively. Similarly, on the USTC-TFC2016 dataset, the $f1$ at 60,000, 90,000, and 120,000 steps are 97.92%, 98.12%, and 98.13%, respectively. Moreover, it can be observed that when the number of pre-training steps reaches 60,000, the classification performance on the ISCXTor2016 and ISCXVPN2016 dataset is already quite impressive.

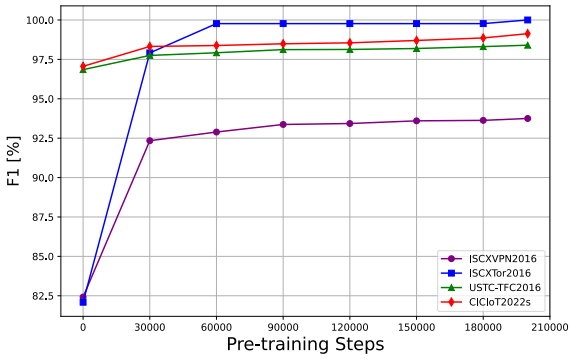
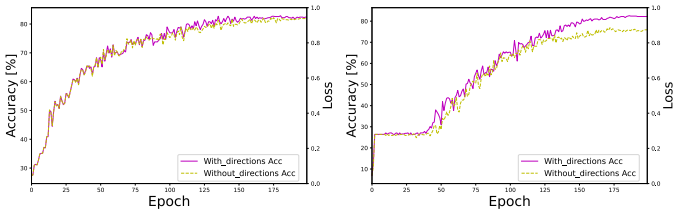


Fig. 7: Performance($f1$) with increasing pre-training steps.

2) Packet Direction: To explore the effect of packet direction on the performance of traffic classification tasks, we conduct experiments on eight datasets (four datasets with direction preservation and other four datasets without direction preservation, i.e., with IP anonymization), with the same model parameter settings. Notably, to eliminate the possible impact of pre-training, all models are trained from scratch without the use of pre-trained weights. The experimental

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

results are shown in Fig. 8. It can be observed that the packet direction has the greatest impact on performance in the ISCXTor2016 dataset, followed by ISCXVPN2016. This suggests that packet direction is an important feature for both of these traffic classification tasks. In contrast, as shown in Fig. 8c and Fig. 8d, the packet direction has minimal influence on both the training process and final performance for the USTC-TFC2016 and CICIoT2022 datasets.



(a) ISCXVPN2016 (b) ISCXTor2016

(c) USTC-TFC2016 (d) CICIoT2022

Fig. 8: Accuracy of models with/without packet direction.

3) Packet Number: In the above experiments, we select the first four packets, each of which takes 256 bytes after the Ethernet layer, that is, the packet header takes 64 bytes and the payload takes 192 bytes. The choice of a 192-bytes payload is a deliberate design decision, a good trade-off. In this section, we vary the number of packets in the input to explore the effect of the number of packets on the performance of BTRFormer. Specifically, we select the first 4, 9, 16, 25, and 36 packets to construct different input configurations, with each packet's byte distribution consistent with the previous description. In this section, the four datasets are used for evaluation. As shown in Table IV, the performance varies across datasets, indicating that the optimal number of packets may differ depending on the traffic characteristics. For example, increasing the number of input packets from 4 to 9 generally improves the $f1$ for the ISCXVPN2016, ISCXTor2016 and CICIoT2022 datasets, indicating that additional packets provide more information. This effect is most pronounced in the ISCXTor2016 dataset, with a gain of 6. 61%. However, on the USTC-TFC2016 dataset, using only the first 4 packets produces better performance, with $f1$ decreasing by 1.19% when using 9 packets. These results suggest that, in some cases, the initial few packets contain sufficient information for effective classification, and including more packets may introduce redundancy or noise.

F. Transfer Learning Experiments

To further evaluate the generalization of the pretrained model, we conduct transfer learning experiments on the ISCXNonVPN2016 and CSTNET-TLS1.3 datasets, which are used exclusively for fine-tuning and evaluation. As shown in Table V, our method achieves an accuracy of 95.26% and 96.84%, respectively, demonstrating strong generalization to new downstream traffic classification tasks.

TABLE IV: $f1$ with varying numbers of packets.

Datasets	4	9	16	25	36
ISCXVPN2016	81.58%	82.65%	80.22%	76.67%	79.78%
ISCXTor2016	82.09%	88.70%	59.19%	49.89%	92.87%
USTC-TFC2016	96.85%	95.66%	85.21%	77.47%	77.65%
CICIoT2022	97.06%	97.63%	96.14%	95.43%	95.73%

TABLE V: Classification performance with pre-training.

Datasets	acc	$f1$
ISCXNonVPN2016	95.26%	95.20%
CSNET-TLS1.3	96.84%	96.83%

G. Efficiency of BTRFormer

To explore the timeliness of traffic analysis, we further evaluate the efficiency of BTRFormer. On our platform, it achieves an average inference time of 4.32 ms per flow, demonstrating its potential for real-time deployment in high-speed networks. For online deployment, it processes consecutive packets within a time window (e.g., 1s, 10s, 20s) for real-time classification.

V. Conclusion and Future Work

Conclusion. In this paper, we propose BTRFormer, a novel Hierarchical Transformer for encrypted traffic classification that leverages AES-inspired 16-byte blocks to construct encryption-aware flow representations. Specifically, each flow is first structured as a traffic matrix of multiple 16-byte blocks, preserving the intrinsic encryption patterns and structural characteristics. We then introduce a Transformer-based architecture with three hierarchical attention layers, enabling the model to effectively capture intra-block, inter-block, and inter-packet dependencies. This multi-level attention mechanism allows BTRFormer to learn fine-grained traffic patterns across the entire flow, resulting in a comprehensive and robust representation. To train the model, we adopt a two-stage strategy: BTRFormer is first pre-trained on large-scale unlabeled traffic to learn latent traffic representations, and subsequently fine-tuned on a small amount of labeled data to adapt to specific classification tasks. Extensive evaluations on four real-world encrypted traffic datasets demonstrate that BTRFormer consistently outperforms state-of-the-art methods.

Future Work. In this study, we focus on block cipher algorithms, which perform encryption/decryption in 16-byte units. Meanwhile, non-block algorithms are increasingly adopted in TLS encryption, such as ChaCha20. To explore the applicability of BTRFormer to such algorithms, we plan to collect traffic data for non-block ciphers and investigate this in future.

Acknowledgment

We are grateful to the anonymous reviewers for their work and insightful feedback. This work was supported by Zhongguancun Laboratory, Beijing, China.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] T. Bujlow, V. Carela-Español, and P. Barlet-Ros, "Independent comparison of popular dpi tools for traffic classification," *Computer Networks*, vol. 76, pp. 75–89, 2015.

[2] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 791–824, 2022.

[3] Z. Wang, K. W. Fok, and V. L. Thing, "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study," *Computers & Security*, vol. 113, p. 102542, 2022.

[4] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, pp. 439–454.

[5] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale." in *NDSS*, 2016.

[6] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 1, pp. 63–78, 2017.

[7] T. Van Ede, R. Bortolameotti, A. Continella, J. Ren, D. J. Dubois, M. Lindorfer, D. Choffnes, M. Van Steen, and A. Peter, "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Network and distributed system security symposium (NDSS)*, vol. 27, 2020.

[8] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, and N. Guizani, "Optimizing feature selection for efficient encrypted traffic classification: A systematic approach," *IEEE Network*, vol. 34, no. 4, pp. 20–27, 2020.

[9] T. Shapira and Y. Shavitt, "Flowpic: Encrypted internet traffic classification is as easy as image recognition," in *IEEE INFOCOM 2019-IEEE conference on computer communications workshops (INFOCOM WKSHPS)*. IEEE, 2019, pp. 680–687.

[10] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "Fs-net: A flow sequence network for encrypted traffic classification," in *IEEE INFOCOM 2019-IEEE Conference On Computer Communications*. IEEE, 2019, pp. 1171–1179.

[11] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2367–2380, 2021.

[12] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.

[13] H. Y. He, Z. G. Yang, and X. N. Chen, "Pert: Payload encoding representation from transformer for encrypted traffic classification," in *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*. IEEE, 2020, pp. 1–8.

[14] X. Meng, C. Lin, Y. Wang, and Y. Zhang, "Netgpt: Generative pretrained transformer for network traffic," *arXiv preprint arXiv:2304.09513*, 2023.

[15] Q. Wang, C. Qian, X. Li, Z. Yao, G. Zhou, and H. Shao, "Lens: A foundation model for network traffic," *arXiv preprint arXiv:2402.03646*, 2024.

[16] S. Guthula, R. Beltiukov, N. Battula, W. Guo, and A. Gupta, "netfound: Foundation model for network security," *arXiv preprint arXiv:2310.17025*, 2023.

[17] T. T. Nguyen and G. Armitage, "A survey of techniques for internet traffic classification using machine learning," *IEEE communications surveys & tutorials*, vol. 10, no. 4, pp. 56–76, 2009.

[18] S. Sen, O. Spatscheck, and D. Wang, "Accurate, scalable in-network identification of p2p traffic using application signatures," in *Proceedings of the 13th international conference on World Wide Web*, 2004, pp. 512–521.

[19] A. W. Moore and K. Papagiannaki, "Toward the accurate identification of network applications," in *International workshop on passive and active network measurement*. Springer, 2005, pp. 41–54.

[20] A. Madhukar and C. Williamson, "A longitudinal study of p2p traffic classification," in *14th IEEE international symposium on modeling, analysis, and simulation*. IEEE, 2006, pp. 179–188.

[21] X. Wang, J. Jiang, Y. Tang, B. Liu, and X. Wang, "Strid^{2}fa: Scalable regular expression matching for deep packet inspection," in *2011 IEEE International Conference on Communications (ICC)*. IEEE, 2011, pp. 1–5.

[22] S. Fernandes, R. Antonello, T. Lacerda, A. Santos, D. Sadok, and T. Westholm, "Slimming down deep packet inspection systems," in *IEEE INFOCOM Workshops 2009*. IEEE, 2009, pp. 1–6.

[23] A. W. Moore and D. Zuev, "Internet traffic classification using bayesian analysis techniques," in *Proceedings of the 2005 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*, 2005, pp. 50–60.

[24] L. Bernaille, R. Teixeira, I. Akodkenou, A. Soule, and K. Salamatian, "Traffic classification on the fly," *ACM SIGCOMM Computer Communication Review*, vol. 36, no. 2, pp. 23–26, 2006.

[25] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, and B. Thuraisingham, "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*, 2016, pp. 177–188.

[26] Y. Chen and Y. Wang, "Mpaf: Encrypted traffic classification with multi-phase attribute fingerprint," *IEEE Transactions on Information Forensics and Security*, 2024.

[27] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 1928–1943.

[28] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[29] K. Lin, X. Xu, and H. Gao, "Tscrnn: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of iiot," *Computer Networks*, vol. 190, p. 107974, 2021.

[30] V. Rimmer, D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," *arXiv preprint arXiv:1708.06376*, 2017.

[31] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: an ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.

[32] X. Wang, Y. Wang, Y. Lai, Z. Hao, and A. X. Liu, "Reliable open-set network traffic classification," *IEEE Transactions on Information Forensics and Security*, 2025.

[33] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proceedings of the ACM Web Conference 2024*, 2024, pp. 1608–1619.

[34] H. Zhang, H. Yue, X. Xiao, L. Yu, Q. Li, Z. Ling, and Y. Zhang, "Revolutionizing encrypted traffic classification with mh-net: A multi-view heterogeneous graph model," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 1048–1056.

[35] X.-Y. Chen, L. Han, D.-C. Zhan, and H.-J. Ye, "Miett: Multi-instance encrypted traffic transformer for encrypted traffic classification," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 15, 2025, pp. 15 922–15 929.

[36] X. Han, G. Xu, M. Zhang, Z. Yang, Z. Yu, W. Huang, and C. Meng, "Degnn: Dual embedding with graph neural network for fine-grained encrypted traffic classification," *Computer Networks*, vol. 245, p. 110372, 2024.

[37] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, and Q. Liu, "Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2066–2075.

[38] T. E. T. Djaidja, B. Brik, S. M. Senouci, A. Boualouache, and Y. Ghamri-Doudane, "Early network intrusion detection enabled by attention mechanisms and rnns," *IEEE Transactions on Information Forensics and Security*, 2024.

[39] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[40] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*. PMLR, 2020, pp. 1597–1607.

[41] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, and Z. Xue, "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5420–5427.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.

[42] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 012–10 022.
[43] C. Wu, F. Wu, T. Qi, and Y. Huang, "Hi-transformer: Hierarchical interactive transformer for efficient and effective long document modeling," *arXiv preprint arXiv:2106.01040*, 2021.
[44] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 16 000–16 009.
[45] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-mae: Leveraging masked autoencoder for accurate, efficient and robust malicious traffic classification," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, 2023, pp. 297–314.
[46] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related," in *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)*, 2016, pp. 407–414.
[47] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *International Conference on Information Systems Security and Privacy*, vol. 2. SciTePress, 2017, pp. 253–262.
[48] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional iot profiling dataset," in *2022 19th Annual International Conference on Privacy, Security & Trust (PST)*. IEEE, 2022, pp. 1–11.
[49] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *2017 International conference on information networking (ICOIN)*. IEEE, 2017, pp. 712–717.
[50] R. Li, X. Xiao, S. Ni, H. Zheng, and S. Xia, "Byte segment neural network for network traffic classification," in *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*. IEEE, 2018, pp. 1–10.
[51] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for dl-based traffic classifier update," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 397–405.
[52] C. Feichtenhofer, Y. Li, K. He *et al.*, "Masked autoencoders as spatiotemporal learners," *Advances in neural information processing systems*, vol. 35, pp. 35 946–35 958, 2022.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:31:38 UTC from IEEE Xplore. Restrictions apply.